

# A Confidentiality-Preserving Distributed Linear Programming Model for Solving Large-Scale Economic Dispatch Problems

Md. Turab Hossain

Department of Computer Science and  
Engineering (CSE)  
Bangladesh University of Engineering  
and Technology (BUET)  
Dhaka, Bangladesh  
turab.cse59@gmail.com

Md. Alamin Hossain

Department of Computer Science and  
Engineering (CSE)  
Bangladesh University of Engineering  
and Technology (BUET)  
Dhaka, Bangladesh  
alamincsebu10@gmail.com

Muhammad Abdullah Adnan

Department of Computer Science and  
Engineering (CSE)  
Bangladesh University of Engineering  
and Technology (BUET)  
Dhaka, Bangladesh  
abdullah.adnan@gmail.com

## Abstract

The primary goal of the economic dispatch (ED) is to organize the generation schedule to fulfill system energy demand at the lowest possible operational cost while meeting all other requirements, including system security and reliability. The inclusion of smart grid technologies requires advanced computational techniques and increased problem-solving capabilities to manage effectively. The in-house computing infrastructure has drawbacks, such as high costs, operational complexity, limited flexibility, and security risks. Even though cloud computing is a substitute, it faces the major challenge of storing sensitive data, such as power system data, in the cloud, which could put organizations at risk of cyberattacks. We propose an outsourcing model of distributed linear programming (LP) that solves large-scale economic dispatch problems. Moreover, for the preservation of data confidentiality, we transform our LP into a confidentiality-preserving LP. We have shown the efficacy of our model by implementing on the real-life Bangladesh power grid system.

## CCS Concepts

• **Computer systems organization** → Cloud computing; Reliability; Availability; • **Security and privacy** → Privacy-preserving protocols; • **Theory of computation** → Linear programming.

## Keywords

economic dispatch; distributed LP; data confidentiality

## ACM Reference Format:

Md. Turab Hossain, Md. Alamin Hossain, and Muhammad Abdullah Adnan. 2024. A Confidentiality-Preserving Distributed Linear Programming Model for Solving Large-Scale Economic Dispatch Problems. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3704522.3704528>



This work is licensed under a Creative Commons Attribution International 4.0 License.

NSysS '24, December 19–21, 2024, Khulna, Bangladesh  
© 2024 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1158-9/24/12  
<https://doi.org/10.1145/3704522.3704528>

## 1 Introduction

Economic dispatch is known as the immediate process of determining the optimal output from several power plants to satisfy system demand at the minimal practical price, given transmission and operating restrictions. The economic dispatch problem is addressed by specialized computer programs. These software must meet the system and operational requirements of the available resources and their related transmission capabilities. The primary principle is that the cheapest group of generators must be utilized first. The marginal cost of the last generating unit required to supply the demand determines the marginal cost of the system. This represents the price of adding a megawatt-hour of electricity to the grid.

From the perspective of system operation, economic dispatch is necessary to dispatch electricity efficiently. Most of the scheduled power comes from generating units, which typically operate seven days a week, twenty-four hours a day. This is known as base-load generation, and it normally involves big coal-fired and nuclear units. The load curve indicates that during the day, the quantity of electricity needed varies; therefore, relying solely on base-load generation is not the most cost-effective alternative. Fossil fuels used in combustion turbines are costlier, as shown in Fig. 1 [2]. In the realm of smart grids, since power flow is becoming two-directional, more and more renewable energy sources will be integrated. In turn, it will make dynamic economic dispatch, data analytics, decentralization, and resilient and efficient energy systems invaluable. Cloud computing will be crucial to the development of smart grids by offering the scalability, flexibility, and infrastructure required to manage the massive volume of data generated and processed in modern energy systems.

Many researchers are paying attention to the economic dispatch issue. Fast Lambda Iteration (FLA) (Zhan et al., [23]), Artificial Neural Network (ANN) (Momoh & Reddy, [15]), Particle Swarm Optimization algorithm (PSO) (Lin et al., [13]), Gravitational Search Algorithm (GSA) (Hota & Sahu, [11]), Mixed Integer Quadratic Programming (MIQP) (Absil et al., [3]), and Flower Pollination Algorithm (FPA) (Vijayaraj & Santhi, [18]) techniques were used to solve ED. However, Jabr et al. [12] presented a simplified security-constrained algorithm in the field of LP applications on ED. Hoke et al. [10] use a rapid and dependable LP technique to calculate the energy density (ED) of grid-tied microgrids. Again, Elsaiah et al. [8] describe a solution for resolving the ED problem in the context of renewable energy sources. Al-Subhi et al. [1] provide an extensive analytical comparison between LP and alternative methods for resolving the economic power dispatch issue. We suggest

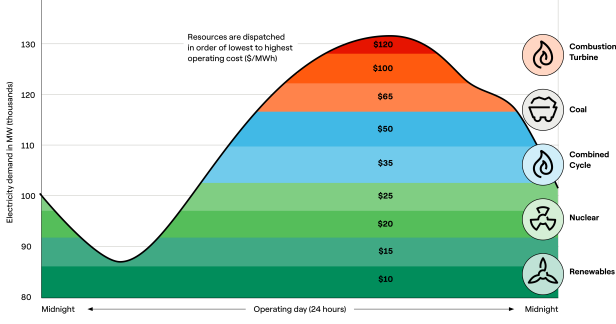


Figure 1: Power plant dispatch order.

a distributed linear programming outsourcing model for solving large-scale economic dispatch problems. As far as we are aware, *no prior work* has considered distributed linear programming for solving large-scale economic dispatch. Our approach is efficient in solving large-scale economic dispatch in power systems.

Moreover, while outsourcing in the cloud, data confidentiality is a must. Leakages in data confidentiality will allow malicious entities to carry out virtual bid assaults and false data injection attacks using network topology data leaks due to breaches in data confidentiality. Since ED is publicly outsourced to the cloud, these challenges [19], [6], and [5] will arise. Various research has focused on preserving data confidentiality. Baek et al. [4] proposed Smart-Frame, an identity-based framework for cloud networks. It is better than conventional public-key cryptosystems as it overcomes scalability problems and saves a large amount of resources for processing and networking. However, this work does not mask the cloud application's mathematical structure, and if a security channel is not established, malicious entities will be able to access the data stream and details about the authentication and authorization mechanism. Again, authors in [14] proposed a model called the PPED model, which ensures the confidentiality of power grid data by using a secure sum protocol. Partitioned data is sent to neighboring nodes. The secure sum protocol allows each node to get the sum value without knowing the individual input. Although they added a new privacy layer, it hides certain functions from generation entities, such as power output, cost functions, and limits. We proposed a model that hides the problem structure as well as the data. In our approach, we transform our linear program (LP) to preserve data confidentiality. The confidential grid data is kept on a local computer system before being masked and safely outsourced to cloud computing. Although Sarker et al. [17] proposed a similar confidentiality-preserving outsourcing model, they did not consider ramp rate, upper bound, lower bound, and other confidential data of generating units.

Our main *objective* is to propose a distributed linear programming approach for solving large-scale economic dispatch. Our model also preserves data confidentiality. Solving an LP for the ED problem for an electricity zone takes about 40 seconds using the lambda iteration method. Generally, ED is solved separately for both real value as well as forecasted value (ex. 5-minute intervals). So to deal with the economic dispatch problem for multiple zones based

on real and forecasted demand and availability, we have to solve a huge number of LPs. Besides time efficiency, as a cloud-based solution, our distributed LP approach will provide fault tolerance, low latency, and cost-effectiveness by ensuring redundancy, optimizing resource allocation, and flexible pricing.

Among this work's principal *contributions* are: i) offering a distributed linear programming outsourcing model for large-scale economic dispatch problems; ii) transforming the traditional ED into a linear program that protects confidentiality and hides the data and problem structure.

We have used the lambda iteration algorithm for solving linear programming. We run our LP programs on top of Apache Spark, which lies between MapReduce and the message-passing interface (MPI) and has closed the gap between industrial and high-performance computing perspectives. We considered our model for both the real-time and day-ahead energy marketplaces. In our model utility, we use data masking to preserve confidential data. We implemented our model on a real power grid system.

Our proposed model is for Independent System Operators (ISO). ISO is responsible for the secure and reliable power dispatch of a region. Generating and distribution entities place their availability and demand on ISO. The corresponding ISO can use our outsourcing approach to solve the economic dispatch problem and dispatch power more efficiently. Outsourcing ED in the cloud will help them improve operational efficiency and focus on strategic growth, all while controlling costs and maintaining flexibility.

The rest of the paper is structured as follows: Cloud computing in power systems and its design are described in Section 2. Section 3 explains the proposed model. Section 4 provides an explanation of the implementation details. The outcomes of the suggested model are described in Section 5. Closing remarks are included in Section 6.

## 2 Cloud Computing for Power Systems

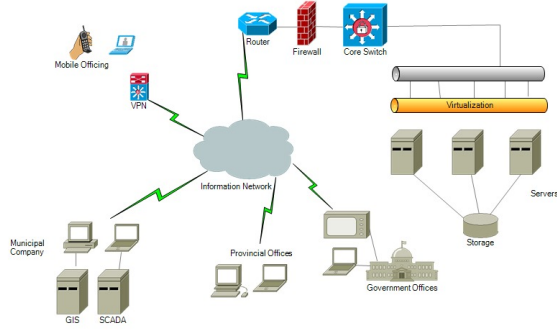
### 2.1 General System Architecture

Deploying a cloud computing data center and distributing applications are the primary tasks involved in creating a new kind of power-dispatching automation system based on cloud computing and enjoying its benefits. Fig. 2 illustrates the important structural elements of the entire system. As the figure shows, the new cloud-based power dispatching system can cover a large area, which includes provincial, municipal, and prefectural firms, in just one cloud computing data center.

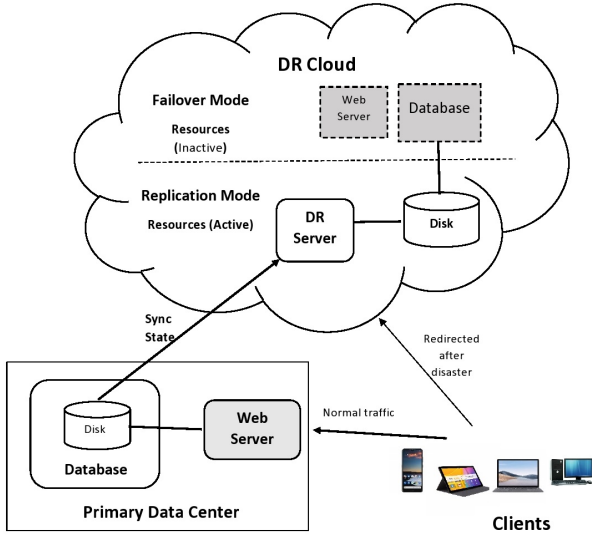
### 2.2 Systems and Networks

IT infrastructure in cloud computing includes foundational hardware, software, networking, and data resources that support the delivery of cloud services. When it comes to storage, distant data replication ought to be carried out via the cloud service's data replication feature because the data is constantly created and modified by the server. In a cloud computing environment, traditional on-premises infrastructure is either supplemented or replaced by virtualized resources that are hosted and managed by cloud service providers (CSPs) like Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure.

Fig. 3 illustrates the basic concept of configuring a disaster recovery system within a cloud context. The disaster recovery system



**Figure 2: Key cloud computing-based power-dispatching system architectures, produced in Network Notepad (Version 1.4.2).**

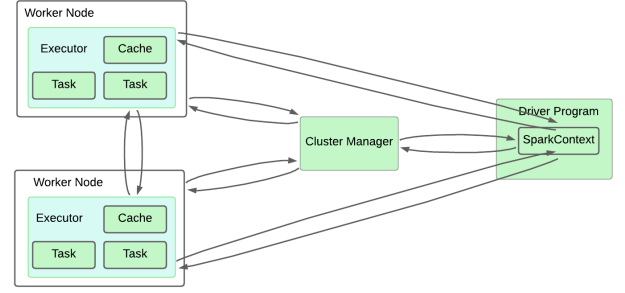


**Figure 3: Cloud-based disaster recovery system conceptual diagram.**

deploys equipment to regularly replicate critical data between the primary and disaster recovery units. Server resources are allocated and utilized as a minimum resource. During a disaster, the disaster recovery center's cloud system uses the resource provisioning function to assign resources to the server and run the recovery system, allowing the client to continue receiving business services.

### 2.3 Cluster Mode

The volume of sheer data that is being generated worldwide is increasing at a greater velocity than ever before. In 2004, Google introduced the MapReduce framework [7] to analyze Big Data to extract knowledge and insight using large-scale (commodity)



**Figure 4: Apache Spark architecture.**

clusters of machines in a distributed way. However, MapReduce does not leverage data persistence well enough, and there is a significant I/O bottleneck in each iteration. On the other hand, in high-performance computing, where objectives are merely high accuracy and fast execution of computations, the current solution for analyzing Big Data is the computational and programming framework known as the message passing interface (MPI) [9]. However, because of the costly infrastructure, MPI is not a feasible option from an industrial point of view. In 2010, Apache Spark generalized MapReduce by introducing Resilient Distributed Datasets (RDD) [22], a distributed memory concept. It bridges the gap between industrial and high-performance computing perspectives and stands between MapReduce and MPI. The main method runs in a process called the Spark driver. The driver schedules tasks for execution following the splitting of the application DAG [20] into stages and then into tasks. In detail, the Spark driver is first connected to the cluster via the SparkContext object. More precisely, SparkContext can establish connections with different types of cluster managers responsible for resource allocations and coordinating tasks. Second, once connected, Spark acquires worker nodes' executors. Executors are workers' processes that are responsible for executing individual tasks and storing data in a given Spark job. Finally, the driver sends the serialized application code to executors to run tasks. Notably, if any worker dies or runs slowly, its tasks will be sent to different executors to be processed again. Fig. 4 shows the cluster mode architecture of Apache Spark.

### 3 Proposed Methodology

Our suggested method's flow diagram is displayed in Fig. 5. It involves a number of steps: collecting raw data, pre-processing it, creating mathematical programming system (MPS) files, using Apache Spark to solve LP, and obtaining the result. The existing raw data of a power system is collected. We separate them into multiple zones. In the pre-processing step, we remove unwanted data and combine it into a structured file format. Linear programming (LP) and mixed-integer problems are displayed and preserved using a file format known as MPS (Mathematical Programming System). Next, we use Apache Spark to solve LP for those MPS files. The following subsections go into more detail regarding these steps:

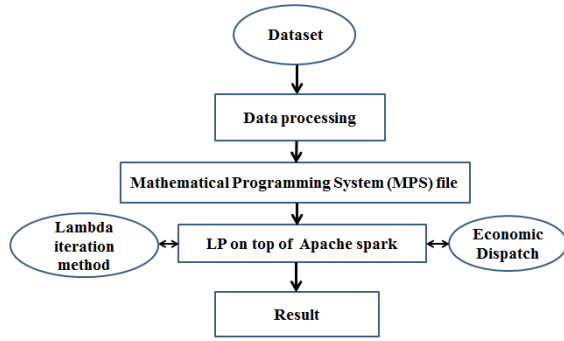


Figure 5: Design flow diagram.

### 3.1 Collection of Data

The dataset is collected from a power system operator utility’s SCADA (Supervisory Control and Data Acquisition) system. Data from both generation and distribution is collected. Regarding economic dispatch, the data of the generation entity is valuable, and its confidentiality must be preserved. The features of the dataset obtained from several generation entities are displayed in Table 1.

| Dataset name      | total zones | total power plants | Fuel type                   |
|-------------------|-------------|--------------------|-----------------------------|
| Generation entity | 9           | 143                | Gas, Coal, HFO, HSD & Solar |

Table 1: Collected dataset.

### 3.2 Data Preprocessing

This method includes organizing the data into a structured file format and eliminating any unnecessary data. After preprocessing, we extract twelve attributes for each power plant in the form Plant name | Recent generation | Cost |  $P_{min}(static)$  | Upper rate | Down rate | T(low) | T(up) | Day availability | Evening availability | Fuel type |  $P_{min}$ . Fig. 6 shows this preprocessed dataset of a zone.

- (1) Plant name: It’s the name of the generating plant.
- (2) Recent generation: The most recent generated value of the power plant.
- (3) Cost: Cost per unit generation from the power plant.
- (4)  $P_{min}(static)$  : The minimum value that a power plant can effectively generate.
- (5) Upper rate: The maximum value that a plant can increase within an hour.
- (6) Down rate: The maximum value that a plant can decrease within an hour.
- (7) T(low): Lowest plant operation time.
- (8) T(up): Highest plant operation time.
- (9) Day availability: Maximum available generation at day period from a power plant.
- (10) Evening availability: Maximum available generation during the evening period from a power plant.

- (11) Fuel type: Types of fuel, such as gas, HSD, HFO, solar, coal, etc.
- (12)  $P_{min}$  : It is the minimum value between  $P_{min}(static)$  and day/evening availability.

### 3.3 MPS File Creation

The MPS (Mathematical Programming System) file format is used to display and preserve mixed-integer and linear programming (LP) problems. This format is supported by almost all commercial LP solvers and the open-source COIN-OR system. Instead of entering the model as equations, MPS is column-oriented, and every model component (variables, rows, etc.) receives names. The value of the NAME record can be anything starting in column 15. Each constraint is identified in the ROWS section; the entries in columns 2 or 3 correspond to equality (E for =) rows, less-than (L for <=) rows, greater-than (G for >=) rows, and non-constraining rows, N. Right-hand-side vectors can be defined in the RHS section, one or more of them. If a variable’s lower and higher boundaries are not given by rows in the matrix, they can be specified in the optional BOUNDS section. Fig. 7 shows a sample mps file for a zone.

### 3.4 LP on Top of Apache Spark

One simple assumption separates vector and matrix operations in Apache Spark, the distributed computing platform for Big Data applications: matrixes must be distributed throughout the cluster, while vectors are local. This assumption has enabled Apache Spark to offer distributed linear algebra and convex optimization. That is, since matrixes are quadratically taking up more storage space than vectors, vector operations should be kept local while matrix operations should be distributed across the cluster [21]. For example, matrix-matrix operations are done over the cluster, and matrix-local vector operations can be efficiently done by broadcasting the local vector to workers containing the matrix partitions. Our primary goal is to evaluate Spark for applications involving large-scale numeric computations, which are more akin to high-performance computing. We use the well-known lambda iteration technique to solve LP. The process continues until the result converges. Thus, large-scale economic dispatch issues can be efficiently solved by solving distributed LP on top of Apache Spark.

### 3.5 Result

Economic dispatch problems can be solved by applying our suggested method in both the real-time and day-ahead marketplaces for energy. We experimented with our model on different types of AWS EC2 instances. We calculate their cost performance. Details of them will be discussed in section 5.

## 4 Implementation details

We utilized an actual power grid dataset. It includes data of 143 power plants. We solved the economic dispatch problem based on real-life data. For simplicity, we do not consider line loss, and we consider the economic cost function to be linear. The details of our implementation will be discussed below.

| Power Plant                        | P(t-1) | Cost  | Pmin | UR  | DR  | T_low | T_up | PmaxDay | PmaxEvening | Fuel | Pmin(static) |
|------------------------------------|--------|-------|------|-----|-----|-------|------|---------|-------------|------|--------------|
| Ghorasal TPP (Unit-1&2)            | 35     | 1.64  | 35   | 85  | 85  | 1     | 24   | 35      | 35          | Gas  | 40           |
| Ghorasal Repowered CCPP Unit-3     | 0      | 1.48  | 0    | 220 | 220 | 1     | 24   | 0       | 0           | Gas  | 100          |
| Ghorasal Repowered CCPP Unit-4     | 0      | 1.46  | 0    | 220 | 220 | 1     | 24   | 0       | 0           | Gas  | 100          |
| Ghorasal TPP Unit-5                | 0      | 1.47  | 0    | 190 | 190 | 1     | 24   | 0       | 0           | Gas  | 100          |
| Ghorasal 365 MW CCPP Unit-7        | 200    | 1.11  | 150  | 365 | 365 | 1     | 24   | 200     | 200         | Gas  | 150          |
| Ghorashal 78.5 MW PP(MAX)          | 70     | 1.69  | 10   | 78  | 78  | 1     | 24   | 70      | 70          | Gas  | 10           |
| Ghorashal 108MW PP (Regent)        | 93     | 1.32  | 10   | 110 | 110 | 1     | 24   | 93      | 93          | Gas  | 10           |
| Haripur GTPP                       | 0      | 2.1   | 0    | 40  | 40  | 1     | 24   | 0       | 0           | Gas  | 10           |
| Haripur 412 MW CCPP                | 412    | 0.98  | 260  | 450 | 450 | 1     | 24   | 412     | 412         | Gas  | 260          |
| Haripur 360MW CCPP(HPL)            | 360    | 1.03  | 185  | 360 | 360 | 1     | 24   | 360     | 360         | Gas  | 185          |
| Meghnaghat 450 MW CCPP(MPL)        | 450    | 1.09  | 260  | 450 | 450 | 1     | 24   | 450     | 450         | Gas  | 260          |
| Meghnaghat 100 MW(IEL)             | 0      | 4.9   | 0    | 100 | 100 | 1     | 24   | 0       | 0           | HFO  | 0            |
| Meghnaghat CCPP(Summit)            | 150    | 1.11  | 100  | 305 | 305 | 1     | 24   | 150     | 150         | Gas  | 100          |
| Madanganj 102 PP(Summit) unit-1    | 80     | 5.37  | 0    | 100 | 100 | 1     | 24   | 80      | 80          | HFO  | 0            |
| Madanganj-55 MW PP(Summit) unit-2  | 55     | 5.29  | 0    | 55  | 55  | 1     | 24   | 55      | 55          | HFO  | 0            |
| Keraniganj 100 MW PP (Powerpac)    | 0      | 9.22  | 0    | 100 | 100 | 1     | 24   | 100     | 100         | HFO  | 0            |
| Narshingdi 22 MW PP (Doreen)       | 22     | 0.757 | 6    | 22  | 22  | 1     | 24   | 22      | 22          | Gas  | 6            |
| 210 MW Siddhirgonj TPP             | 0      | 2.58  | 0    | 210 | 210 | 1     | 24   | 0       | 0           | Gas  | 100          |
| Siddhirgonj 2*120 MW GTPP          | 0      | 1.73  | 0    | 210 | 210 | 1     | 24   | 0       | 0           | Gas  | 60           |
| Siddhirgonj 100 PP(Dutch Bangla)   | 0      | 4.9   | 0    | 100 | 100 | 1     | 24   | 0       | 0           | HFO  | 0            |
| Siddhirgonj 335 MW CCPP            | 330    | 1.03  | 120  | 335 | 335 | 1     | 24   | 330     | 330         | Gas  | 120          |
| Gagnagar 102 MW PP (Digital Power) | 0      | 4.86  | 0    | 102 | 102 | 1     | 24   | 0       | 0           | HFO  | 0            |

Figure 6: Preprocessed dataset.

```

NAME      THREEZONES
ROWS
N COST
L LIM1
G LIM2
E MYEQN
COLUMNS
X001 COST 0.82
X001 MYEQN 1
X002 COST 1.02
X002 MYEQN 1
X003 COST 9.51
X003 MYEQN 1
X004 COST 16.4
X004 MYEQN 1
X005 COST 9.62
X005 MYEQN 1
X006 COST 8.77
X006 MYEQN 1
DHKA COST 5.77
DHKA MYEQN 1
SYLH COST 3.77
SYLH MYEQN 1
RAJH COST 8.77
RAJH MYEQN 1
RHS
RHS1 MYEQN 750.0
BOUNDS
UP BND1 X001 150.0
UP BND1 X002 20.0
UP BND1 X003 95.0
UP BND1 X004 20.0
UP BND1 X005 200.0
UP BND1 X006 100.0
UP BND1 DHKA 150.0
UP BND1 SYLH 150.0
UP BND1 RAJH 100.0
ENDATA

```

Figure 7: Created MPS file.

#### 4.1 Formulation of the Confidentiality-Preserving Linear Program

To solve the ED, we can first think of the simplest form of an LP.

$$\min c^T x \quad (1)$$

where  $c^T$  is the cost vector and  $x$  is the decision variable. The desired function is dependent upon

$$Ax \leq b \quad (2)$$

$$x \geq 0 \quad (3)$$

Where  $A$ , the constraint coefficient matrix, has  $M$  by  $V$  elements (i.e.,  $R^{M \times V}$ ) and with  $M$  elements (i.e.,  $R^{M \times 1}$ ), the right-hand side (RHS) column vector is denoted by  $b$ .  $V$  indicates the total amount of decision variables in the LP, and  $M$  represents the number of requirements. The variable coefficients of the generator outputs and voltage angles are stored in matrix  $A$ . The generator, line, and voltage limitations, as well as the demand for each bus, are stored in  $b$ . To maintain data security, the coefficients of LP ( $c^T$ ,  $A$ , and  $b$ ) are masked. This is done by locally producing a random diagonal monomial matrix  $D$  with  $M$  by  $M$  entries (i.e.,  $R^{M \times M}$ ) (considering the matrix dimension) so that the overall equation cannot be changed. Now the conventional LP is turned into a confidentiality-preserving LP by multiplying all coefficients of the LP by the random matrix  $D$ . So the confidentiality-preserving LP can be written in this form.

$$\min c^T D x \quad (4)$$

such that

$$D A x \leq D b \quad (5)$$

$$x \geq 0 \quad (6)$$

Notably, the resultant outputs (the value of  $x$ ) are not hidden because a cyberattacker cannot reveal the meaning of each variable without knowing what's underneath in  $A$  and  $b$ . The most optimal result obtained with a confidentiality-preserving LP is identical to that obtained using standard LP because, even when scaled (4)–(6),

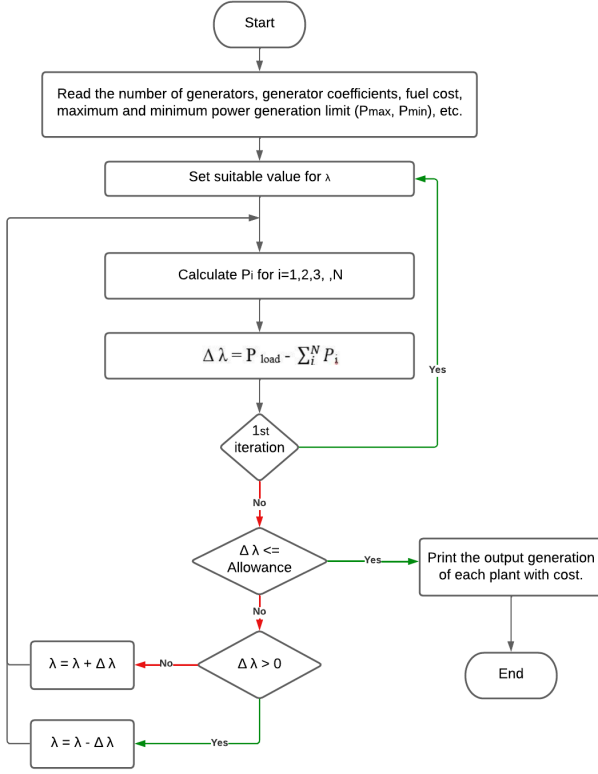


Figure 8: Lambda iteration method.

the objective function gives the same (1)–(3) suitable result. Thus, while executing and transferring confidentiality-preserving LP, the system operator (SO) only publishes the secured matrixes DA, Db, and  $c^T D$ . As a result, the original data in  $c^T$ , A, and b gets secured, as potential cyber attackers can't get them without knowledge of the random diagonal monomial matrix D. To remove the inequality constraints, we need to consider slack variables, as described in [17].

## 4.2 Lambda Iteration Method

To solve economic load dispatch with linear programming, we used the lambda iteration method [16]. For solving LP, the lambda iteration method is a conventional method. It is an iterative type of computational technique shown in fig. 8. The optimum operating point of any generator set within a specified limit is found using this method.

## 4.3 Apache Spark Cluster

We solved our LP on top of Apache Spark. We used Spark version 1.6.1 and Scala 2.11.x. When we pass our MPS file to the MPS file parser, it converts the raw data of the MPS file into LP standard form. Thus, it constructs two 2D matrixes, which are inequalities constraints coefficients and equalities constraints coefficients. Again, it generates four 1D matrixes of objective function: inequalities

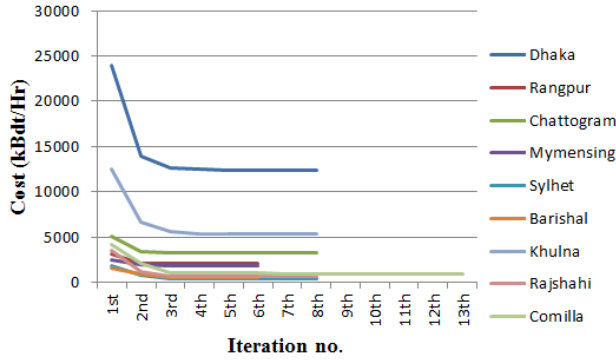
constraints limits, equalities constraints coefficients, equalities constraints limits, lower bounds and upper bounds. Since matrixes take quadratically more storage space than vectors, we distribute the cost function, equality constraints coefficient, and equality constraints limits into some number of partitions. On the other side, we consider vectors to be local. Any Spark application has its own Directed Acyclic Graph (DAG), consisting of RDDs (Resilient Distributed Dataset) as vertices and transformations and actions as edges. When the Spark driver wants to evaluate an action, it divides the DAG into stages at the DAG-Scheduler, and more importantly, optimization happens through pipelining the transformations (lazy evaluation of transformations plays a crucial role here). Then, each stage is further divided into tasks at the task scheduler, where actual computations start to happen. A function that distributes the matrix and solves LP is shown in Algorithm 1. For the cluster resources, we have used Amazon's Elastic Computing instances with pre-installed and configured Apache Spark 1.6.1.

### Algorithm 1 MPSLinearProgram Solver

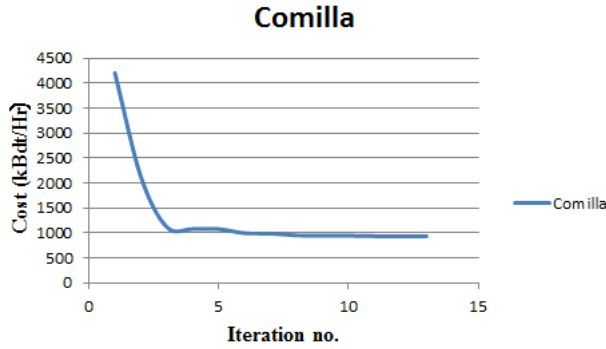
- 1: **Input:**
- 2: *datasetParser*: This object parses and provides data components required for the LP.
- 3: *numPartitions*: Number of partitions to divide the data into for distributed processing across the Spark cluster
- 4: **Output:**
- 5: *optimalIds*: A vector of Plant IDs to operate for optimal demand
- 6: **Initialize:**
- 7: *sparkContext* ← new *SparkContext*()
- 8: *converter* ← new *LPStandardConverter.toStandardForm*(  
*datasetParser.getC*, *datasetParser.getG*, *datasetParser.getH*,  
*datasetParser.getA*, *datasetParser.getB*, *datasetParser.getLb*,  
*datasetParser.getUb*)
- 9: *dVector* ← *sparkContext.parallelize*(*converter.getStandardC.toArray*, *numPartitions*).*glom.map*(new *DenseVector*())
- 10: *dMatrix* ← *sparkContext.parallelize*(*converter.getStandardA.toArray.transpose.map*(*Vectors.dense().toSparse* :  
*Vector*), *numPartitions*)
- 11: *densevector* ← new *DenseVector*(*converter.getStandardB.toArray*)
- 12: (*optimalVal*, *optimalX*) ← *LP.solve*(*dVector*, *dMatrix*, *dMatrix*,  
*sparkContext* = *sparkContext*)
- 13: *optimalIds* ← *Vectors.dense*(*optimalX.flatMap.toArray.collect*())
- 14: **return** *optimalIds*

## 4.4 Deployment

Our cluster is deployed on Amazon EMR. Amazon EMR enables teams to handle enormous volumes of data rapidly, cost-effectively, and at scale. S3 buckets are used for the file system. We made use of many Ec2 instance types. General-purpose instances such as m4.large and m4.xlarge were used. Compute-optimized Ec2 instances, such as c4.large and c4.xlarge, were also used. Then, we evaluate each other's performances. The following section will go into further depth about this.



(a) LP convergence graph for all zones.



(b) LP convergence graph for a particular zone.

Figure 9: LP convergence graph.

## 5 Results

### 5.1 LP Convergence for ED Problem

For solving LP, we have used the lambda iteration method. It's an iterative type of computational technique. LP convergence for all zones and a particular zone is shown in Fig. 9a and Fig. 9b. The vertical axis shows the total cost, kBdt/Hr, while the horizontal axis shows the number of iterations. We found that after the 3rd iteration, we get very much closer to the ED result, so setting the total iteration count to 10 will be enough to solve the ED.

### 5.2 Solving Economic Dispatch

We solve the economic dispatch (ED) to satisfy the system demand at the least probable cost. The primary principle involves prioritizing the generators with the lowest marginal costs to fulfill the load at the lowest possible total cost. The marginal cost required by the last generator to fulfill the load is the system's marginal cost, which is the cost of adding an extra one MWh of power to the system. To calculate the features related to the input and output of power plants, this remarkable technique for economic dispatch was introduced to manage power stations that rely on fossil fuels. While solving ED, we considered important generation parameters like per unit fuel generation cost, last hour generation value, upper rate, down rate, duration of scheduling, maximum day availability, evening availability, effective zone, forbidding zone, fuel type,

etc. We deployed and ran our model on top of Apache Spark in the Amazon EMR. Table 2 displays the solution of an economic dispatch problem for a zone after running it into the cloud.

| Plant ID                          | Output (in MW)         | Plant ID | Output (in MW)         |
|-----------------------------------|------------------------|----------|------------------------|
| 1                                 | 79.99                  | 13       | 99.99                  |
| 2                                 | 220.99                 | 14       | $2.89 \times 10^{-10}$ |
| 3                                 | 359.99                 | 15       | 21.99                  |
| 4                                 | 359.99                 | 16       | 10.99                  |
| 5                                 | $9.14 \times 10^{-14}$ | 17       | $1.73 \times 10^{-11}$ |
| 6                                 | 54.99                  | 18       | 32.99                  |
| 7                                 | 4.99                   | 19       | $1.02 \times 10^{-10}$ |
| 8                                 | 7.99                   | 20       | 19.99                  |
| 9                                 | 34.99                  | 21       | $2.71 \times 10^{-11}$ |
| 10                                | $2.39 \times 10^{-10}$ | 22       | $2.25 \times 10^{-10}$ |
| 11                                | $9.17 \times 10^{-14}$ | 23       | 90.00                  |
| 12                                | $9.68 \times 10^{-11}$ | 24       | $1.83 \times 10^{-10}$ |
| Total time elapsed: 67.85 seconds |                        |          |                        |

Table 2: Generation Output of Comilla Zone.

### 5.3 Computational Analysis for Outsourcing Framework

We perform our experiments on different cloud instances (m4.large, m4.xlarge, c4.large, and c4.xlarge). The cloud and local infrastructure are summarized in Table 3.

| EC2 type  | vCPU | ECU | Memory (Gib) | Linux/Unix usage |
|-----------|------|-----|--------------|------------------|
| m4.large  | 2    | 6.5 | 8 GB         | \$0.10/hr        |
| m4.xlarge | 4    | 13  | 16 GB        | \$0.20/hr        |
| c4.large  | 2    | 8   | 3.75 GB      | \$0.10/hr        |
| c4.xlarge | 4    | 16  | 7.5 GB       | \$0.199/hr       |

Table 3: Computational analysis.

The performance comparison between these AWS instances is shown in Fig. 10. The Y-axis corresponds to the time in seconds, and the X-axis corresponds to the number of zones. It shows that m4.large takes the most time, while c4.xlarge is the fastest among them. It's been found that c4.large can't solve the LP problem for zone counts greater than 6. Since its physical memory is the smallest among them, it causes out-of-memory errors while solving LP for more than six zones.

Fig. 11 illustrates the loss or performance gain for every cloud object. Overall, for the trials, the average time was calculated, and then the changes in the percentage were obtained against the calculation with the average time for m4.xlarge. The m4.xlarge is compared to the c4.large, m4.large, and c4.xlarge group of instances from Amazon EC2, and the performance change was observed at -21.5%, -39.2%, and +14.08%, respectively. So it was found that C4 family instances are more efficient than M4 instances because the C4 family is compute-optimized, having high-performing processors, while the M4 family is considered as general-purpose processors. Ordinary ED problems usually benefit from high-performing CPUs rather than requiring extensive RAM like dynamic grid architectures.

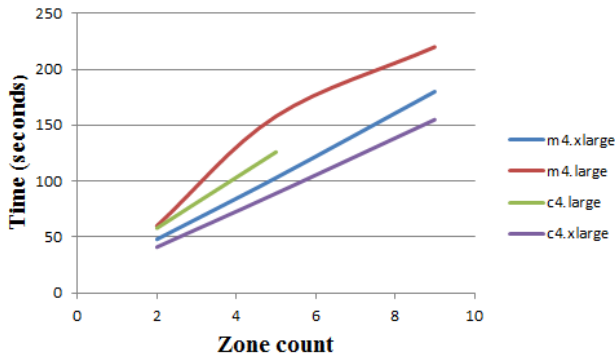


Figure 10: Performance comparison between m4.xlarge, m4.large, c4.large and c4.xlarge for cluster size 3 (2 core, 1 master).

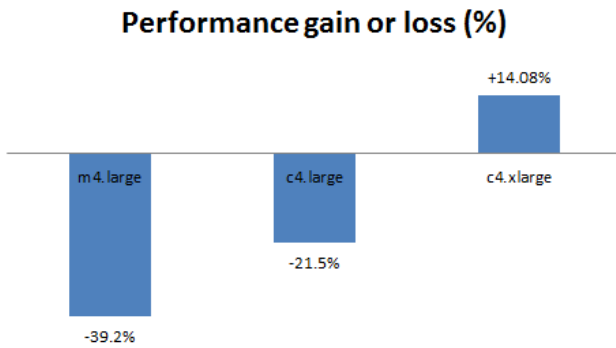


Figure 11: Performance gain or loss (%). Percentage change was calculated considering average duration for computation of m4.large, c4.large, and c4.xlarge against the average computation time of m4.xlarge instances.

## 6 Conclusion and future work

Cloud computing opens up a wide range of opportunities for cost-effective operations of high-resource-requiring grid applications. Power system operators (SO) can achieve greater operational efficiency, enhance security, reduce costs, and support the integration of renewable energy by adopting cloud computing. We have developed a large-scale economic dispatch solver, solving LP on top of the Apache Spark optimization module. Our approach supports efficient, sparse, and dense linear algebra computations.

At present, we don't consider transmission losses. We will expand our methodology in the future to address unit commitment problems, ED problems involving transmission losses, and other nonlinear energy system optimization problems.

## References

- [1] Ahmad Al-Subhi and Hesham Alfares. 2016. Economic Load Dispatch Using Linear Programming: A Comparative Study. *International Journal of Applied Industrial Engineering* 3 (Jan 2016), 16–36. <https://doi.org/10.4018/IJAIE.2016010102>
- [2] Enel North America. 2023. what is a peaking power plant. <https://www.enelnorthamerica.com/insights/blogs/what-is-a-peaking-power-plant>. Accessed: 2024-08-17.
- [3] Michael Azzam, S. Easter Selvan, Augustin Lefèvre, and P. A. Absil. 2014. Mixed Integer Programming to Globally Minimize the Economic Load Dispatch Problem With Valve-Point Effect. *arXiv:1407.4261 [math.OC]*
- [4] J. Baek, Q. H. Vu, J. K. Liu, X. Huang, and Y. Xiang. 2015. A Secure Cloud Computing Based Framework for Big Data Information Management of Smart Grid. *IEEE Transactions on Cloud Computing* 3, 2 (April 2015), 233–244. <https://doi.org/10.1109/TCC.2014.2359460>
- [5] D. Choi and L. Xie. 2018. Economic Impact Assessment of Topology Data Attacks With Virtual Bids. *IEEE Transactions on Smart Grid* 9, 2 (March 2018), 512–520. <https://doi.org/10.1109/TSG.2016.2535246>
- [6] Dae-Hyun Choi and Le Xie. 2013. Ramp-Induced Data Attacks on Look-Ahead Dispatch in Real-Time Power Markets. *IEEE Transactions on Smart Grid* 4, 3 (2013), 1235–1243. <https://doi.org/10.1109/TSG.2012.2228508>
- [7] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. *Commun. ACM* 51 (Jan 2004), 137–150. <https://doi.org/10.1145/1327452.1327492>
- [8] Salem Elsaiah, Mohammed Benidris, Joydeep Mitra, and Niannian Cai. 2014. Optimal economic power dispatch in the presence of intermittent renewable energy sources. *IEEE Power and Energy Society General Meeting* 2014 (July 2014). <https://doi.org/10.1109/PESGM.2014.6939903>
- [9] Message P Forum. 1994. MPI: A Message-Passing Interface Standard.
- [10] A. Hoke, A. Brissette, S. Chandler, A. Pratt, and D. Maksimović. 2013. Look-ahead economic dispatch of microgrids with energy storage, using linear programming. In *2013 1st IEEE Conference on Technologies for Sustainability (SusTech)*. 154–161. <https://doi.org/10.1109/SusTech.2013.6617313>
- [11] Prakash Hota and Nakul Sahu. 2015. Non-Convex Economic Dispatch with Prohibited Operating Zones through Gravitational Search Algorithm. *International Journal of Electrical and Computer Engineering (IJECE)* 5 (Dec 2015), 1234–1244. <https://doi.org/10.11591/ijece.v5i6.pp1234-1244>
- [12] R. A. Jabr, A. H. Coonick, and B. J. Cory. 2000. A homogeneous linear programming algorithm for the security constrained economic dispatch problem. *IEEE Transactions on Power Systems* 15, 3 (Aug 2000), 930–936. <https://doi.org/10.1109/59.871715>
- [13] Jie Lin, C.-L. Chen, S.-F. Tsai, and Chengqing Yuan. 2015. New intelligent particle swarm optimization algorithm for solving economic dispatch with valve-point effects. *Journal of Marine Science and Technology (Taiwan)* 23 (Feb 2015), 44–53. <https://doi.org/10.6119/JMST-013-1231-2>
- [14] Avikarsha Mandal. 2016. Privacy Preserving Consensus-Based Economic Dispatch in Smart Grid Systems. In *Future Network Systems and Security*, Vol. 670. Springer International Publishing, 98–110. [https://doi.org/10.1007/978-3-319-48021-3\\_7](https://doi.org/10.1007/978-3-319-48021-3_7)
- [15] J.A. Momoh and S. Reddy. 2014. Combined Economic and Emission Dispatch using Radial Basis Function. *IEEE Power and Energy Society General Meeting* 2014 (July 2014), 1–5. <https://doi.org/10.1109/PESGM.2014.6939506>
- [16] Zaineb Nisar. 2021. Economic Load Dispatch using Lambda Iteration, Particle Swarm Optimization and Genetic Algorithm. *International Journal for Research in Applied Science and Engineering Technology* 9 (Aug 2021). <https://doi.org/10.22214/ijraset.2021.37527>
- [17] M. R. Sarker, J. Wang, Z. Li, and K. Ren. 2018. Security and Cloud Outsourcing Framework for Economic Dispatch. *IEEE Transactions on Smart Grid* 9, 6 (Nov 2018), 5810–5819. <https://doi.org/10.1109/TSG.2017.2697430>
- [18] S. Vijayaraj and R. K. Santhi. 2016. Multi-area economic dispatch using flower pollination algorithm. In *2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*. 4355–4360. <https://doi.org/10.1109/ICEEOT.2016.7755541>
- [19] L. Xie, Y. Mo, and B. Sinopoli. 2011. Integrity Data Attacks in Power Market Operations. *IEEE Transactions on Smart Grid* 2, 4 (Dec 2011), 659–666. <https://doi.org/10.1109/TSG.2011.2161892>
- [20] Yinggen Xu, Liu Liu, and Zhijun Ding. 2020. DAG-Aware Joint Task Scheduling and Cache Management in Spark Clusters. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 378–387. <https://doi.org/10.1109/IPDPS47924.2020.00047>
- [21] Reza Bosagh Zadeh, Xiangrui Meng, A. V. Ulanov, Burak Yavuz, Li Pu, Shivaram Venkataraman, Evan R. Sparks, Aaron Staple, and Matei A. Zaharia. 2015. Matrix Computations and Optimization in Apache Spark. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2015). <https://api.semanticscholar.org/CorpusID:207238816>
- [22] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-tolerant Abstraction for In-memory Cluster Computing. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation* (San Jose, CA) (NSDI'12). USENIX Association, Berkeley, CA, USA, 2–2. <http://dl.acm.org/citation.cfm?id=2228298.2228301>
- [23] J. P. Zhan, Q. H. Wu, C. X. Guo, and X. X. Zhou. 2014. Fast  $\lambda$ -Iteration Method for Economic Dispatch With Prohibited Operating Zones. *IEEE Transactions on Power Systems* 29, 2 (March 2014), 990–991. <https://doi.org/10.1109/TPWRS.2013.2287995>